

2025 Semester 1
CITS4404: Artificial Intelligence and Adaptive Systems

Project Deliverable 1

Group 19

Vinayak Gupta 24066272
Shu-Chi Liu 23831023
Zixiao Ma 24116864
Gerard Newman 22705329
Jordan Rigden 22496593
Lewei Xu 23709058

Algorithm 1: Particle Swarm Optimization

By Shu-Chi Liu

The paper proposes a concept of optimizing nonlinear functions using a particle swarm approach, known as the Particle Swarm Optimization (PSO) algorithm. The failure of previous algorithms primarily stemmed from the inherent limitations of bio-inspired evolutionary algorithms, as exemplified by genetic algorithms, which relied excessively on mechanistic transplantation of biological metaphors. Specifically, operations such as chromosome encoding, crossover, and mutation introduced artificial complexity—for instance, requiring specially designed path encoding schemes when solving the Traveling Salesman Problem (TSP)—while also suffering from widespread issues like premature convergence and high computational costs.

The development of the Particle Swarm Optimization (PSO) algorithm originated from the biomimetic simulation of bird flock behaviour, with its evolution reflecting the refinement and optimization of algorithm design. The initial version simulated bird movement by combining nearest-neighbour velocity matching and a “crazy factor” for random perturbations. Later, the introduction of the “cornfield vector” mechanism established a dual-guidance system based on personal best positions (pbest) and global best positions (gbest). As the algorithm progressed, researchers eliminated redundant elements (such as the crazy factor) and focused on the collaborative optimization of individual memory and collective knowledge. The original symbolic judgment was replaced by distance-based velocity adjustments, ultimately leading to a streamlined and efficient version where both individual and global guidance were unified using a random coefficient with a mean of 1.

The theoretical foundation of PSO aligns closely with Millonas’ five principles of swarm intelligence: proximity, quality response, diversity, stability, and adaptability. Its uniqueness lies in treating candidate solutions in the search space as particles with momentum. Through the dual guidance of pbest and gbest, combined with a random perturbation mechanism, PSO ensures convergence efficiency while effectively avoiding local optima. Experimental studies have shown that PSO performs comparably to the backpropagation algorithm in neural network training tasks (such as the XOR problem and Fisher’s Iris classification) and even exhibits superior generalization performance (e.g., achieving a 3% improvement in accuracy in EEG classification tasks). In challenging optimization problems (such as the Schaffer f6 function), PSO performs on par with genetic algorithms, reliably finding the global optimum.

As a bridge between evolutionary computation and neural processing, PSO integrates the randomness of evolutionary programming with the information exchange concepts of genetic algorithms. Its core innovation lies in optimization search through the dynamic flight of particles in hyperspace. This mechanism naturally balances exploration and exploitation: random factors promote the exploration of unknown regions, while the momentum effect from velocity inheritance enables intelligent “overshooting” toward target regions. This interdisciplinary algorithm, which blends perspectives from social psychology and engineering optimization, not only validates the optimization essence of collective behaviour in nature but also embodies the philosophical concept of “emergent intelligence” — the

emergence of complex intelligent behaviour from simple interaction rules. While further research is needed in aspects such as parameter adaptation and application boundaries, PSO has already demonstrated remarkable problem-solving capabilities and vast application potential.

References

- J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of the IEEE International Conference on Neural Networks (ICNN)*, Perth, Australia, 1995, pp. 1942–1948.

Algorithm 2: Artificial Bee Colony Optimization (ABC)

By Lewei Xu

A numerical value optimizer is used for various optimization problems such as searching, routing, assignment, and task allocation. It is known for its effectiveness in addressing unconstrained and constrained optimization problems, as well as multi-dimensional numeric problems in the field of computer science. Based on the collection and processing of nectar: each bee decides to reach the nectar source by following a nestmate who has already discovered a patch of flowers. It integrates a local search strategy with a random search strategy.

The algorithm mimics the foraging behaviour of honey bees, bees are split into 3 groups: Employed bees, Onlookers and Scouts. Scout bees search for new food sources (or solutions in the hypothesis space) and share the fitness information with the Onlooker bees. The Onlooker bees choose their food source (of best solution) based on the probability of nectar amount, this food source is replaced if Scout bees find a better source. Employed bees are responsible for a specific food source (solution), they exploit their assigned food source (solution) and search for nearby/neighbouring solutions using small modifications. Keep the better solution if it is better than the old one. If the food source (solution) is no longer improving after multiple iterations, the solution may be abandoned. Onlooker bees also have the role of observing food sources (solutions) and choosing which ones to exploit/optimize further.

Summary:

- Employed Bees – each bee is assigned to a known food source (solution) and works to improve it. If the new solution is better, it replaces the old one.
- Onlooker Bees – evaluate food sources based on their nectar amount (fitness value) and probabilistically choose the best ones to exploit further.
- Scout Bees – if a food source is no longer improving, it is abandoned, and a scout bee randomly explores the search space to find a new solution.

Steps:

1. Initialization – Randomly generate a set of potential solutions (food sources).
2. Employed Bee Phase – Improve assigned solutions through local search.
3. Onlooker Bee Phase – Select and refine promising solutions probabilistically.
4. Scout Bee Phase – Replace poor solutions with new random ones.
5. Repeat – The process iterates until a stopping criterion is met.

Scout bees are the randomness component of the algorithm, by randomly selecting new solutions, we are more likely to find a **global** maxima/minima instead of just the local. Employed bees implement a local search strategy (e.g. Powell's/Hook-Jeeves).

ABC was developed to address challenges in function optimization and combinatorial problems, particularly when traditional algorithms struggle with higher dimensional

optimization problems. ABC is a derivative-free method, unlike gradient-based algorithms. Provides a better balance between global search and local search. ABC seeks to solve problems that other algorithms seem to have difficulty with. For example, GA often converges on a local optima instead of the global optima, whilst PSO lacks a mechanism to abandon unproductive search regions, leading to slower convergence. SA often does inefficient exploration due to random search mechanisms. ABC solves these issues by using a. 3-phase division of tasks (3 types of bees) to find local and global optima and abandon unproductive search regions.

ABC was used to generate solutions for various computation problems such as the traveling salesman problem and ride-matching problem and was shown to produce results that were very close if not exactly the optimal values. ABC was found to outperform or match the performance of GA and PSO in most test cases, it is more effective in real-world optimization and high-dimensional spaces. The paper provides strong evidence that BCO is a better alternative to existing optimization techniques. The paper claims that BCO has improved convergence, adaptability, and robustness are well-supported by experimental results. Computational cost can be higher in some cases due to its random scout bee phase, which may introduce inefficiencies and computation overhead.

References:

- Karaboga, D., Basturk, B. A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm. J Glob Optim 39, 459–471 (2007). https://doi.org/10.1007/s10898-007-9149-x

Algorithm 3: Bacterial Foraging Optimisation (BFO)

By Jordan Rigden

Bacterial Foraging Optimisation (BFO) is an algorithm inspired by the natural foraging behaviour (specifically, chemotaxis) of the bacterium Escherichia coli. E. coli utilises several strategies to enhance its search for nutrients, ensuring its survival and reproduction. These strategies include chemotaxis, swarming, reproduction, and elimination-dispersal processes.
At the core of E. coli's survival strategy is its chemotactic movement — the way an individual bacterium navigates its environment. E. coli displays a pattern of search and avoidance behaviour, performing two key actions: "run" and "tumble." When the bacterium detects an increasing nutrient gradient, it performs a "run," moving straight toward the favourable area. Conversely, when there is no gradient or the environment becomes less favourable, it performs a "tumble," reorienting itself to explore new directions. While both actions occur regardless of external conditions, the bacterium adjusts the duration of each to favour movement towards richer environments. This sophisticated behaviour is governed by a simple form of memory, a set of control mechanisms, chemical sensing, and the bacterium's ability to move.

In addition to individual movement, bacterial colonies display a behaviour known as swarming. This involves bacteria being drawn towards others located in higher-energy environments, guided by the chemical attractants they release. Swarming patterns vary among different bacterial species, each offering unique advantages for survival.

Bacteria that thrive in nutrient-rich environments are more likely to reproduce, increasing the population in these favourable areas. Following reproduction, each bacterium continues its chemotactic search, potentially discovering even better environments. Interestingly, bacteria are described as "never satisfied" — they continuously tumble in search of improved conditions. In optimisation terms, this behaviour expands the search within promising areas of the solution space, increasing the chances of discovering better solutions. However, it also carries the risk of converging prematurely on local optima, which might not represent the global best solution.

Finally, the processes of elimination and dispersal play a critical role in bacterial foraging. Bacteria in low-resource environments are likely to be eliminated, while dispersal events — caused by factors like water currents or transport by animals — relocate bacteria to new areas, which might be more or less resource-rich. Depending on the conditions of the new environment, bacteria will either proliferate or die off. This natural mechanism helps the population explore diverse regions of the solution space, enhancing the likelihood of finding optimal solutions.

BFO is a highly exploratory technique that allows a thorough and variable search through the solution space. Although a current optimal area may have been found, it still attempts to explore the solution space through techniques such as chemotaxis and dispersal events. This helps to break out of local optima if relevant and find the best global answer. In the context of a share trading bot, the market trends and data may be very noisy and irrational. This may create instances of share trading that work in the short term but may not be

applicable in the long term. BFO aims to escape local optima through its chemotactic behaviour and thoroughly search the solution space. The main difference in BFO is the chemotactic movement of each bacterium. The general movement in the direction of a "high resource" area, while still implementing the tumbling movement, allows for the exploration of many areas in the solution space and thus the opportunity to avoid local optima. This movement differs from most other nature-inspired optimisation techniques and will prove important for the analysis of our use case. The method of this paper replicates the optimisation technique by defining each effect to create a cost function formula. It then gives a pseudo-biological example of how these parts of the function affect the solution search space. However, the paper does not provide a use case for the optimisation technique with verifiable results, and thus I referred to another article to validate this technique.

I found the article “Bacteria foraging optimization algorithm based load frequency controller for interconnected power system” by Ali et al. Here, they use BFO to optimise the parameters of a Proportional-Integral (PI) controller that automates the control to maintain stability in a power system and minimise oscillations. The paper by Ali et al. found that BFO was a superior parameter optimisation technique when compared to a conventional PI controller and one tuned by a Genetic Algorithm. The study concluded that BFO was able to find a better combination of parameters and thus provide a more stable frequency with fewer oscillations than the other algorithms. The authors claimed that this was a superior approach and had the potential for integration into real-world control of power distribution in this use case.

References:

- K. M. Passino, "Biomimicry of bacterial foraging for distributed optimization and control," in IEEE Control Systems Magazine, vol. 22, no. 3, pp. 52-67, June 2002, doi: 10.1109/MCS.2002.1004010.
- E. S. Ali and S. M. Abd-Elazim, "Bacteria foraging optimization algorithm based load frequency controller for interconnected power system," *International Journal of Electrical Power & Energy Systems*, vol. 33, no. 3, pp. 633–638, Mar. 2011, doi: 10.1016/j.ijepes.2010.12.022.

Algorithm 4: Grey Wolf Optimisation Algorithm (GWO)

By Vinayak Gupta (24066272)

The Grey Wolf Optimizer (GWO) is a population-based metaheuristic algorithm inspired by the social hierarchy and group hunting behaviour of grey wolves (Canis lupus). It was introduced as an improvement over traditional optimisation techniques such as Genetic Algorithms (GA), Particle Swarm Optimization (PSO), Differential Evolution (DE), Evolutionary Programming (EP), Evolution Strategy (ES), and Gravitational Search Algorithm (GSA). These earlier methods, while effective in some scenarios, often face challenges such as premature convergence, sensitivity to parameter settings, and an inability to balance exploration and exploitation—particularly when dealing with complex, multimodal, and high-dimensional problems.

GWO addresses these limitations by modelling the leadership structure found in wolf packs. Individuals in the population are assigned roles—alpha, beta, delta, or omega—based on fitness. The top three solutions (alpha, beta, and delta) guide the position updates of the remaining wolves, simulating the encircling and hunting of prey. The algorithm uses coefficient vectors (A and C) and a control parameter (a) that decreases over time to help balance exploration and exploitation dynamically. GWO stands out for its simplicity, requiring minimal parameter tuning compared to other algorithms, while still delivering strong performance.

The novelty of GWO lies in how it integrates biological inspiration with a mathematically elegant model. It mimics the wolves' tracking, attacking, and social behaviours using a set of equations, allowing the population to explore broadly in the early stages and converge toward promising solutions later. The algorithm was validated through experiments on 29 benchmark test functions, including unimodal, multimodal, and composite types. Each function was tested 30 times to ensure statistical reliability. Metrics such as average fitness, standard deviation, and convergence speed were used to evaluate its performance.

In addition to benchmark testing, GWO was applied to three real-world engineering design problems: tension/compression spring design, welded beam design, and pressure vessel design. It was also tested in optical engineering for optimising photonic crystal waveguide parameters. In most cases, GWO achieved optimal or near-optimal solutions, showing competitive or superior results compared to GA, PSO, DE, EP, ES, and GSA.

The paper provides comprehensive information, including mathematical formulations, pseudo-code, parameter settings, and links to code, making the algorithm highly reproducible. These factors, combined with its low complexity, make GWO a practical tool for solving challenging real-world problems.

In conclusion, the authors' claims about GWO being simple, effective, and robust are well supported by experimental evidence. Its ability to maintain a good balance between exploration and exploitation, coupled with its ease of implementation, makes it suitable for a wide range of optimisation problems. Given its strong performance and adaptability, in my

opinion, GWO is especially promising for dynamic applications like AI-based trading bots, where solution quality and reliability are essential.

References

- S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey Wolf Optimizer," Advances in Engineering Software, vol. 69, pp. 46–61, 2014.

Algorithm 5: Ant Colony Optimization(ACO)

By Zixiao Ma(24116864)

The Ant Colony Optimization (ACO) algorithm addresses critical limitations observed in traditional optimization methods such as genetic algorithms (GA) and simulated annealing (SA), particularly in solving combinatorial optimization problems. Conventional approaches often suffer from premature convergence in high-dimensional discrete spaces, such as the Traveling Salesman Problem (TSP), due to rigid evolutionary operators like crossover and mutation in GA or static probabilistic criteria in SA. These methods also introduce artificial complexity through specialized encoding schemes, such as path representations, which complicate implementation. Additionally, their reliance on fixed parameters results in an inefficient balance between exploration and exploitation. ACO circumvents these challenges by emulating the decentralized, pheromone-mediated coordination of ant colonies, enabling adaptive search dynamics in complex environments.

Prior algorithms faltered in discrete optimization tasks due to inherent structural constraints. For instance, GA's crossover and mutation operators require cumbersome encoding schemes that obscure problem-specific heuristics, while SA's lack of long-term memory traps it in local optima, hindering global exploration. Gradient-based methods, though effective in continuous spaces, fail in combinatorial domains like scheduling or routing due to their dependence on differentiable functions. ACO overcomes these shortcomings through stigmergy—a form of indirect coordination via pheromone trails—which dynamically adjusts search behaviour based on collective experience, eliminating the need for rigid evolutionary mechanisms.

The core innovation of ACO lies in its pheromone feedback system, which probabilistically guides artificial ants to construct solutions using both pheromone intensity and heuristic information (e.g., proximity in TSP). Global pheromone updates reinforce high-quality solutions while evaporation prevents stagnation, balancing exploration through probabilistic path selection and exploitation via pheromone accumulation. Key advancements include the MAX-MIN Ant System, which imposes adaptive pheromone bounds to avoid premature convergence, and ACO-R, a hybrid extension for continuous optimization that integrates discretization strategies.

The efficacy of ACO is demonstrated through rigorous validation across diverse benchmarks. In TSPLIB instances, ACO achieved tour lengths 7% shorter than GA for the "eil51" dataset. Job scheduling simulations showed a 12% reduction in makespan compared to SA, while ACO-R delivered competitive error rates on continuous functions like Rosenbrock when tested against particle swarm optimization (PSO). The study provides detailed pseudocode, parameter configurations, and reproducibility guidelines, ensuring transparency and replicability.

Results highlight ACO's superior convergence, solving TSP in 30% fewer iterations than GA, and robustness in dynamic environments, such as job scheduling with stochastic delays. Scalability is evidenced by consistent performance in high-dimensional problems, including 200-node TSP instances. Validation via statistical analyses—30 independent runs—and

direct comparisons with GA, SA, and PSO confirmed improvements in solution quality and computational efficiency.

While the authors' conclusions are partially justified—ACO's stigmergic mechanism excels in balancing exploration-exploitation—its limitations include significant computational overhead from pheromone maintenance (20% slower than PSO in continuous tasks) and dependency on problem-specific heuristics, such as distance matrices, which restrict plug-and-play applicability. Although ACO thrives in combinatorial optimization, adapting it to continuous domains like trading bot parameter optimization necessitates hybrid frameworks like ACO-R, increasing implementation complexity. Consequently, despite its strengths in discrete spaces, ACO's suitability for the trading project is limited, as its core advantages are diluted when transformed for continuous parameter tuning.

References

- Dorigo, M., & Di Caro, G. (1999). Ant Colony Optimization: A New Meta-Heuristic. IEEE Transactions on Evolutionary Computation.
- Stützle, T., & Hoos, H. H. (2000). MAX-MIN Ant System. Future Generation Computer Systems.

Algorithm 6: Whale Optimisation Algorithm

By Gerard Newman (22705329)

The Whale Optimisation Algorithm (WOA) is a nature-inspired metaheuristic optimization algorithm that attempts to mimic the hunting strategy of humpback whales. The algorithm begins by generating a population of agents (whales) located at candidate solutions, that through iterations move around and explore the search space. The iterations of the algorithm can then be understood by breaking it down into its exploration and exploitation modes. During exploitation, there are two mechanisms that the whales can swap between, these are the “shrinking encircling mechanism” and the “spiral updating position mechanism,” both of these modes are focused on the current best-found solution. There is a 50% chance of choosing between either of these exploitation mechanisms while exploiting. On the other hand, during exploration, the whales will explore randomly based on the position of each other. This allows for a wide and thorough search of the space. The algorithm begins with more of the whales dedicated to exploration and as time passes more will be dedicated to exploitation.

The problems with existing algorithms that the Whale Optimisation Algorithm attempts to solve are premature convergence, getting trapped in local maxima, and suboptimal exploration-exploitation balance. Previous attempts have failed to solve these problems sufficiently for all classes of problems that are commonly faced, partly due to the large variety of search spaces that can arise. Most search algorithms tend to be most efficient on a subset of problems. Of those algorithms that do perform well across a large variety of problems, another problem arises, these algorithms tend to be highly complex and come with a high number of meta parameters. Ideally, a search algorithm should be simple to implement and easy to tune to the problem.

The new idea introduced by this algorithm is to combine a combination of encircling and spiralling motion around the current best solution, in a sense perpendicular to what the gradient direction would be expected to be rather than to move in a gradient-like direction. In this way more information can be gathered about the space around the best solution and different types of common optimisation traps can be avoided. By using such a simple exploitation method, the number of tuneable meta parameters is reduced to only two, and the implementation of the algorithm is simpler than those existing in the literature.

The paper demonstrates the effectiveness of this new method by applying it to 35 common benchmark problems, 29 of these problems being mathematical optimization problems and 6 being structural design problems. The results from the WOA are then compared to the results of other algorithms already existing in the literature, namely against the Particle Swarm Optimisation (PSO), Gravitational Search Algorithm (GSA), Differential Evolution (DE) and Fast Evolutionary Programming (FEP) optimization algorithms. The comparisons presented in the paper indicate that the WOA performs very competitively, often achieving results significantly better than those in the literature. The algorithm appears to avoid premature convergence and falling for local minima.

The authors conclude that the WOA is a promising algorithm with an efficient optimization technique with improvements over traditional search algorithms. While the results presented look good for the WOA, leading the authors to claim a positive result, the problems and the algorithms selected for comparison could have easily been cherry-picked to make the WOA look favourable. As such, more comparisons and more problems should be explored to get a better picture of the algorithm's performance.

References

- Mirjalili, S. and Lewis, A. (2016) 'The whale optimization algorithm', *Advances in Engineering Software*, 95, pp. 51–67. doi:10.1016/j.advengsoft.2016.01.008.

Conclusions

All the reviewed methods are variations on the main optimisation technique of swarm intelligence. The initial algorithm Particle Swarm Intelligence is the basic technique from which each paper we reviewed was built.

Artificial Bee Colony Optimisation (ABC), Grey Wolf Optimisation (GWO, 2014), and Whale Optimisation (WOA, 2016) are inspired by Particle Swarm Optimisation (PSO, 1995) to varying degrees, but they are not merely improvements of PSO. They introduce entirely new heuristic rules and strategies. These methods inherit the core concepts of swarm intelligence, but each has innovated based on different animal behaviors. For example, GWO simulates the hunting behavior of grey wolf packs, while WOA mimics the predatory behavior of whales. These methods address the issue of early PSO getting trapped in local minima by introducing various stochastic mechanisms. By achieving a more refined balance between exploration and exploitation, they enable the population of candidate solutions to converge more effectively toward the global optimum.

Similarity:
These six algorithms — Ant Colony Optimization (ACO), Artificial Bee Colony (ABC), Particle Swarm Optimization (PSO), Whale Optimization Algorithm (WOA), Grey Wolf Optimizer (GWO), and Bacterial Foraging Optimization (BFO) — are all bio-inspired metaheuristic algorithms, meaning:

- They are suitable for solving NP-Hard optimization problems like feature selection.
- They do not require gradient information, which makes them robust in complex, nonlinear, and high-dimensional spaces.
- They provide a good balance of exploration and exploitation, which is vital for avoiding local optima and ensuring global search capability.

These shared characteristics make them ideal for the feature selection task. All the algorithms researched are a type of swarm intelligence algorithm, meaning that they mimic the group behaviour of certain species or microscopic organisms. We selected these algorithms as they are all stochastic in nature, meaning that they balance exploration and exploitation to search globally for an optimal solution instead of converging on the first local optima found.

Algorithm	**Year Introduced**	**Inspired by**	**Exploration Mechanism**	**Exploitation Mechanism**
Particle Swarm Optimisation	1995	Social behaviour of bird flocks	It is achieved through the random movement of particles in the solution space.	In PSO, exploitation is embodied in the convergence of particles toward their personal best (pBest) and the global best (gBest) positions.
Artificial Bee Colony	2005	Foraging behaviour of	Scout bees search randomly around	Employed bees try to find better

Optimisation		honey bees	where current bees are located	solutions using currently known solutions.
Bacterial Foraging Optimisation	2002	Foraging behavior of E. coli bacteria	Random motion and random elimination, with random repositioning of good solutions to explore the entire search space	Bacteria in good locations will reproduce more than those in poor locations, leading to an increased concentration around the best solutions
Grey Wolf Optimisation	2014	Leadership hierarchy in grey wolves	Achieved by random position updates influenced by the top three wolves (alpha, beta, delta) to explore broadly.	As iterations progress, the control parameter ‘a’ decreases, guiding wolves to converge around the best solutions.
Ant Colony Optimisation	2004	Ant colony foraging behaviour	ACO diversifies search via probabilistic path selection (balancing pheromones and heuristics) and pheromone evaporation, preventing stagnation by weakening outdated trails.	It refines solutions by reinforcing pheromones on high-quality paths and leveraging heuristic data (e.g., proximity in TSP) to prioritize optimal routes.
Whale Optimisation	2016	The bubble net hunting strategy of humpback whales	Random motion around the search space with the position of other whales in consideration	Whales spiral around and encircle the current best solution